

中国科学院大学

《计算机组成原理(研讨课)》实验报告

姓名 唐菁 学号 2024K8009970006 专业 计算机科学与技术
实验项目编号 4-2 实验名称 功能型处理器结构优化-高速缓存设计

- 注 1: 撰写此 Word 格式实验报告后以 PDF 格式保存 SERVE CloudIDE 的 /home/serve-ide/cod-lab/reports 目录下(注意: reports 全部小写)。文件命名规则: prjN.pdf, 其中 prj 和后缀名 pdf 为小写, N 为 1 至 4 的阿拉伯数字。例如: prj1.pdf。PDF 文件大小应控制在 5MB 以内。此外,实验项目 5 包含多个选做内容,每个选做实验应提交各自的实验报告文件,文件命名规则: prj5-projectname.pdf,其中“-”为英文标点符号的短横线。文件命名举例: prj5-dma.pdf。具体要求详见实验项目 5 讲义。
- 注 2: 使用 git add 及 git commit 命令将实验报告 PDF 文件添加到本地仓库 master 分支,并通过 git push 推送到实验课 SERVE GitLab 远程仓库 master 分支(具体命令详见实验报告)。
- 注 3: 实验报告模板下列条目仅供参考,可包含但不限定如下内容。实验报告中无需重复描述讲义中的实验流程。

一、基本思路说明、具体设计细节

1.1 时序状态

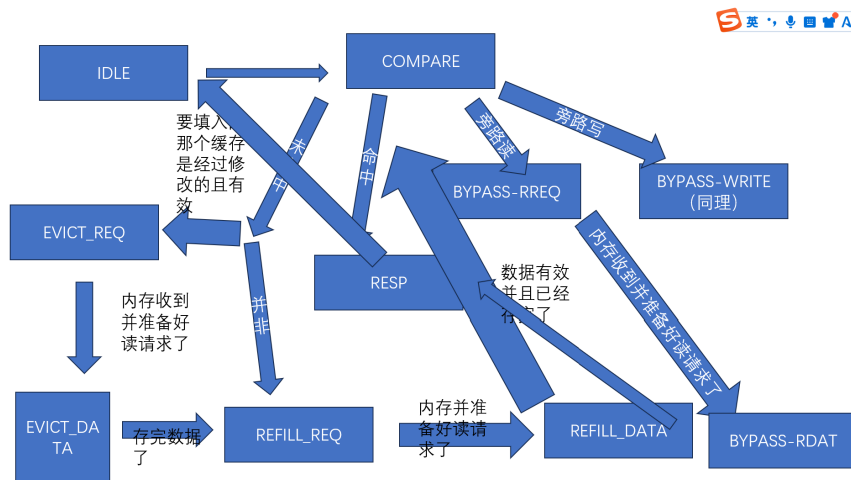


图 1: DCACHE 设计:时序框图

图看着很乱,但是我们可以进行适当地拆解和路径分析。

首先,当 cache 处于 Idle 状态下,如果 cpu 发出了一条有效的请求内存访问指令,那么就进入 compare 状态。Compare 状态下:

1. 如果是旁路,即地址范围在 0x00_0x1F 或 $\geq 0x4000_0000$,那么往往是外设等等不经过缓存的一些地址,所以走旁路读/写。如果对设备发出收到指令,准备读/写之后,进入获取/写入 data 的阶段。而当应答(读/写)完成之后,就进入 RESP 阶段,准备将对应的数据给 cpu(读),或者直接进入空闲状态(写)
2. 如果是直接命中的,即缓存中直接就有我们想要的的数据,那么直接获取对应数据(读)或者直接写入相应数据(写),然后准备回到 RESP
3. 如果没有命中,那么就需要从内存中拿对应地址的数据写回缓存。如果缓存中对应的位置的数据是经过 CPU 修改过的,为了防止覆盖,那么一定要先写回内存,进入 EVICT 阶段。

4. EVICT 进行完后(或者根本不用 EVICT 阶段),就直接进入重填部分。重填部分是将内存对应的数据写回缓存/将缓存对应的数据写回内存。然后内存准备好之后,就写入/读出 DATA,操作有效并且 8 个字节全部写入/读出后,就进入 RESP 状态。

ICACHE 由于其指令寄存器,所以不写。同时没有旁路这件事,一般来说。

1.2 读写时序描述

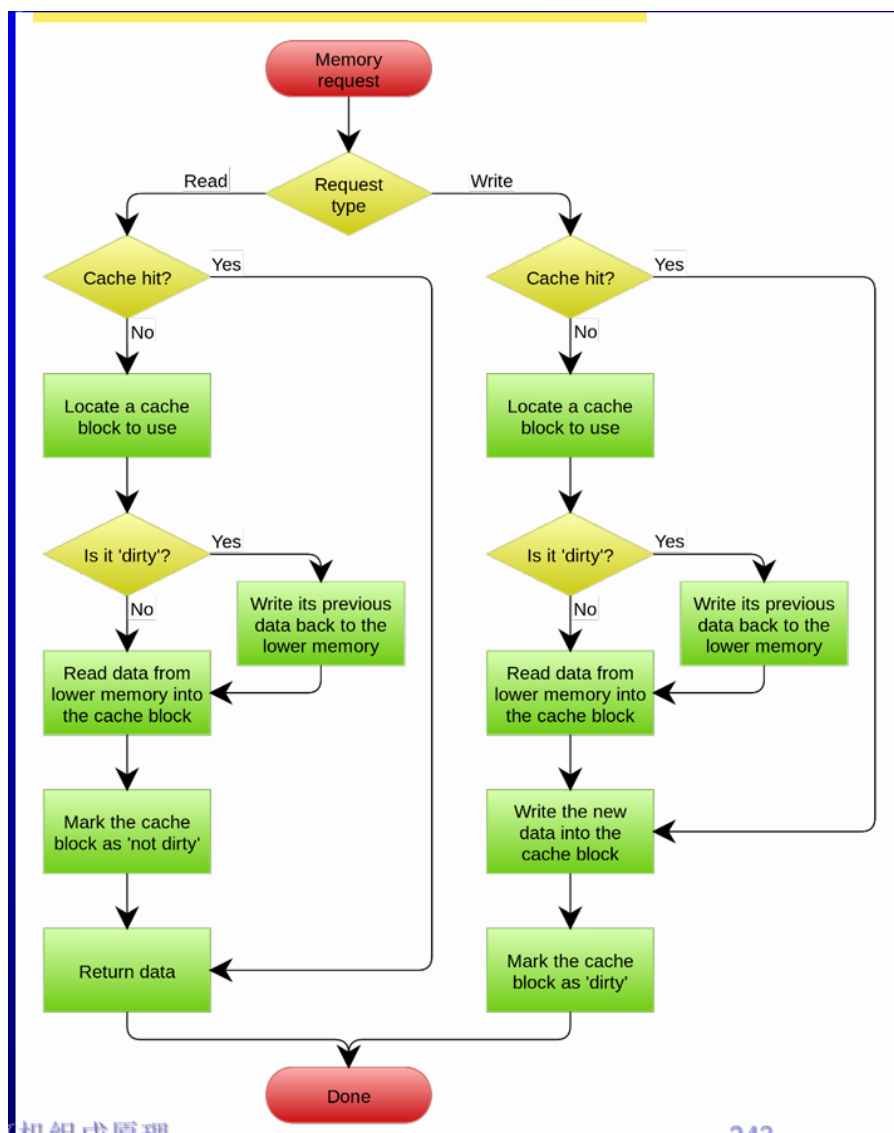


图 2: 写的流程图(C09 课件上的流程图)

1. 写如果命中,那么直接将 CPU 的数据写到缓存对应的位置上

如果没有命中,那么先将对应地址上的内存块写到缓存对应的位置上。这个写会带来问题:如果这个位置上已经被 CPU 修改过,会带来覆盖,那么应该先将这个块写回到内存。

然后,再将 CPU 的数据写到缓存对应的位置上。

2. 读如果命中,直接向 CPU 返回数据

如果没有命中,那么需要将对应地址的数据从内存搬进来(如果需要替换,和上文逻辑一致),下一轮肯定会命中了。

1.3 Cache 的结构说明

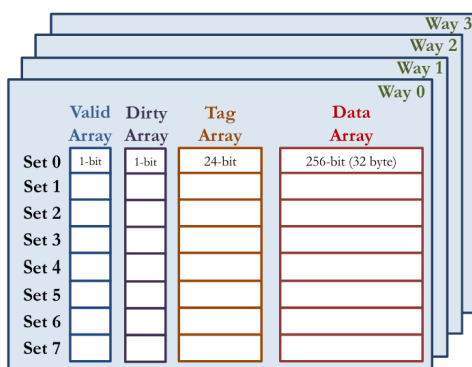


图 3: 数据 cache 内部结构

cache 分作 3 个维度:4 个道和 8 组,交叉出 32 个数据点。然后每个数据点 32 个 byte,8 个 4 字节数据。

data_cache 中,每个数据点不止有 data_array 的 32 个字节,还有 tag_array,用于在命中的时候和 cpu 的地址的高 24 位进行比较,观察是否命中、是否缓存中存的是我们想要的数。同时还有 dirty_array,用于告诉我们这个数据块是否被 cpu 修改过(表示来自 cpu)。如果修改过,下一次还要覆盖重填,那么就必须要存回内存,防止遗失。然后是 valid_array,用于表明这个数据是否是来自于内存的有效数据。

1.4 各个小逻辑

1. 突发写逻辑

如果是旁路,是不需要突发写的,长度为 1 的情况即可

如果不是旁路,那么是需要突发写的。因为 cache 的数据需要一整块全部写进去,一共 8 个字,所以需要 burst_cnt 寄存器,每次数据有效写入之后增 1,从 0 到 7 之后就写入结束。

2. 地址逻辑

```
1 wire [4:0] req_offset = req_addr_latch[4:0];
2 wire [2:0] req_index = req_addr_latch[7:5];
3 wire [23:0] req_tag = req_addr_latch[31:8];
4 wire [2:0] req_word_idx = req_offset[4:2]; // 包含 8 个字, 32个字节
```

其中最高的 24 位是和 cpu 所需要的地址进行比较,如果是相等,说明是对应的数据。

req_index 标志的是 8 个 set 中的哪一个

req_word_idx 标明的是这一个字块里面的哪一个字

为什么要这样设计地址? 让我们正好来复习一下理论课的内容吧

首先,一个字块 8 个字,每个字 4 个字节,所以一共占 5 位字块内地址,即 req_offset

然后,一共 8 个组,所以 3 位组间地址,即 req_index

剩下的,就是表明是哪一个大组的 tag: 就是剩下的 24 位地址,用于比较是否是我们所需要的地址:req_tag

3. 选择 WAY 的逻辑

只要是这一个组的,就可以写入随机的一个 WAY,那么这个 WAY 的选择,我们用一个 replace_cnt 来表明:

```
1 reg [1:0] replace_cnt [0:`CACHE_SET-1];
```

[1:0] 表示 4 个道,一共 8 个寄存器,分别表明 8 组中替换哪一个

初始化: 当 rst 的时候都赋值成为 0

增 1: 每一次从内存填入数据结束 (rsp_last) 之后,就在对应的组中加 1。这样可以让 4 路都充分地使用。而且恰好可以用位宽是 4 位来实现溢出之后直接取余:即 3->4 之后,直接选择第 0 路,因为 5 溢出了。

```
1 REFILL_DATA: begin
2   if (from_mem_rd_rsp_valid) begin
3     data_array[req_index][evict_way][burst_cnt] <= from_mem_rd_rsp_data;
4     burst_cnt <= burst_cnt + 1'b1;
5     if (from_mem_rd_rsp_last) begin
6       replace_cnt[req_index] <= replace_cnt[req_index] + 1'b1;
7
8   end
9   end
```

二、 实验中遇到的 Bug 等问题的解决和思考

1. replace_cnt 增加 1 的时机问题出现了这样的问题:

```
1 ERROR: at 00ns.
2 Yours: PC = 0x00000090, rf_waddr = 0x0f, rf_wdata = 0x00000000
3 Reference: PC = 0x00000090, rf_waddr = 0x0f, rf_wdata = 0x00000001
```

说明是向寄存器写入的数据不对。根据波形图,得出当前的指令是:lw x15 0(x9)

这个数据来自于缓存/内存,说明缓存/内存中的数据不对。我们看到当前的地址是 F4. 我们看前面谁的地址是 F4. 所以我就开始看之前几拍的 instruction:

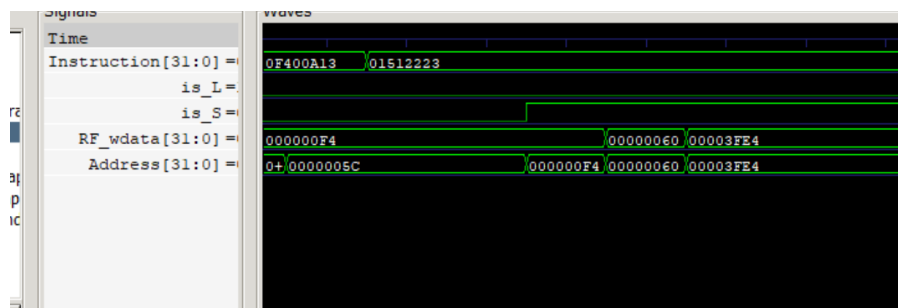


图 4: Instruction: 对 F4 地址的操作

此时可以看出: 机器码 (Hex): 01512223,

sw x21, 4(x2)

对 F4 进行了写入(即将 x21 寄存器内容写进去), 说明多半就是写入出问题了。

然后分析代码逻辑得出, evict_way 在 replace_cnt 增 1 后才赋值, 即:

```
1 wire [1:0] evict_way = replace_cnt[req_index];
```

但是增 1 之后, 原先应该进行替换的路没有写回内存, 直接被新的数据覆盖, 导致之前对内存内容的修改根本就没有返回到内存, 导致后续再用这个数据的时候出现了错误。

应该先赋值 evict_way, 然后再增 1, 这样我们就将应该写回的数据写回了。即增 1 时刻: 应该在整个 Cache Miss 处理完成(重填结束)时, 再去增加轮询指针。

即:

```
1  REFILL_DATA: begin
2  if (from_mem_rd_rsp_valid) begin
3  data_array[req_index][evict_way][burst_cnt] <= from_mem_rd_rsp_data;
4  burst_cnt <= burst_cnt + 1'b1;
5  if (from_mem_rd_rsp_last) begin
6  replace_cnt[req_index] <= replace_cnt[req_index] + 1'b1;
7
8  end
9  end
```

2. 当时出现了一个卡死状态,代码直接进入了无限循环,出现了 15GB 的波形文件。这是因为 Cache 没有锁存 CPU 的请求地址,导致 req_ready 和 req_valid 同时为 1 之后,CPU 就会认为请求已经被接收,并在下一拍撤销或改变 from_cpu_inst_req_addr,状态机就很容易错乱,进入一些无限循环

三、性能计数器的比较

我们选取 load-store 这样和内存交互进行较多的 matrix-mul 进行测试。

首先,当使用 cache 的时候,我们可以看到:

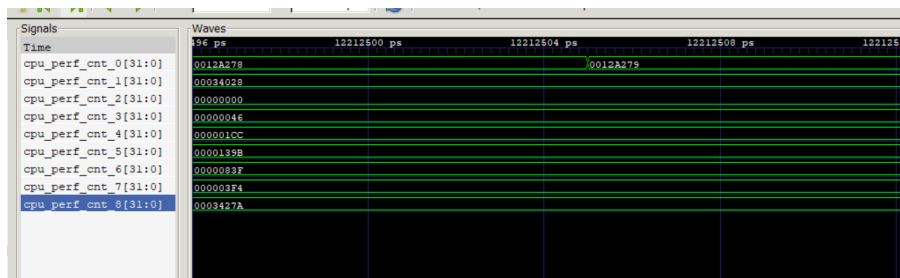


图 5: 使用 cache 时 matrix-mul 的性能计数器,在结束时的值

然后,当我们不使用 cache 的时候,我们可以看到:

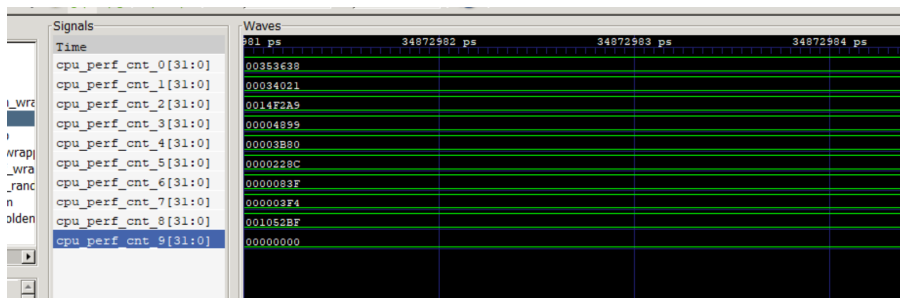


图 6: 不使用 cache 时 matrix-mul 的性能计数器,在结束时的值

我们列一个表格,进行分析:

表 1: 无 Cache 与有 Cache 性能计数器对比表

计数器名称	功能说明	无 Cache (Hex)	无 Cache (Dec)	有 Cache (Hex)	有 Cache (Dec)
cpu_perf_cnt_0	总的周期数	00353638	3,487,288	0012A278	1,221,240
cpu_perf_cnt_1	总的完成指令数	00034021	213,025	00034028	213,032
cpu_perf_cnt_2	IF 处打转周期数	0014F2A9	1,372,841	00000000	0
cpu_perf_cnt_3	ST 原地打转	00004899	18,585	00000046	70
cpu_perf_cnt_4	LD 原地打转	00003B80	15,232	000001CC	460
cpu_perf_cnt_5	RDW 原地打转	0000228C	8,844	0000139B	5,019
cpu_perf_cnt_6	总的 load 次数	0000083F	2,111	0000083F	2,111
cpu_perf_cnt_7	总的 store 次数	000003F4	1,012	000003F4	1,012
cpu_perf_cnt_8	IW 阶段打转周期数	001052BF	1,069,759	0003427A	213,626

我们可以看到,有了 cache 之后,总的周期数大幅度减少。其中 icache 起到极大作用,IW 处几乎打转等待指令少了很多;对 data 的 load-store 影响也极大,在 ST 和 LD 和 RDW 处打转周期数也少了很多。

四、 对讲义中思考题(如有)的理解和回答

1. 问题:如何实现大于 4 byte 的指令/数据交互?

Cache 只在传输开始的第一拍发送一个起始地址以及突发长度

内存接收到起始地址后,其内部的地址计数器会自动递增。在接下来的 7 个时钟周期内,主从双方无需再次交互地址,只需在数据线上连续传送剩余的 7 个字(共 28 字节)。

当 burst_cnt == 3'b111(即传输第 8 个字)且握手成功时,总线会触发一个 last 信号。检测到 last 信号后,状态机得知整条大于 4 字节的数据(32 字节)已安全交互完毕,状态机随后转向下一状态(如 RESP 或 IDLE),并拉低总线请求。

2. :如果用上述接口只传输 32-bit 数据(len = 0),last 在哪个位置拉高?

答:last 信号会在第一拍(也是唯一的一拍)数据传输的同时拉高。只有当 valid 和 ready 同时为高(即双向握手成功)的那个时钟上升沿,这单拍的数据才被成功接收,同时 last 信号也在此刻生效。

五、 实验所耗时间

在课后,你花费了大约 3 小时完成此次实验。